```
define_methods

counter       # => 0
inc(4)
counter       # => 4
```

This example defines two *Kernel Methods (32)*. (It also uses *Dynamic Dispatch (49)* to access the private class method define_method on Kernel.) Both Kernel#counter and Kernel#inc can see the shared variable. No other method can see shared, because it's protected by a *Scope Gate (81)*—that's what the define_methods method is for. This smart way to control the sharing of variables is called a *Shared Scope*.

Spell: Shared Scope

Shared Scopes are not used much in practice, but they're a powerful trick and a good example of the power of scopes. With a combination of Scope Gates, Flat Scopes, and Shared Scopes, you can twist and bend your scopes to see exactly the variables you need, from the place you want. Now that you wield this power, it's time for a wrap-up of Ruby closures.

## Closures Wrap-Up

Each Ruby scope contains a bunch of bindings, and the scopes are separated by *Scope Gates (81)*: class, module, and def.

If you want to sneak a binding or two through a Scope Gate, you can use blocks. A block is a *closure*: when you define a block, it captures the bindings in the current environment and carries them around. So you can replace the Scope Gate with a method call, capture the current bindings in a closure, and pass the closure to the method.

You can replace class with Class.new, module with Module.new, and def with Module#define_method. This is a *Flat Scope (83)*, the basic closure-related spell.

If you define multiple methods in the same Flat Scope, maybe protected by a Scope Gate, all those methods can share bindings. That's called a *Shared Scope (84)*.

Bill glances at the road map he created. (See *Today's Roadmap*, on page 74.) "Now that you've gotten a taste of Flat Scopes, we should move on to something more advanced: instance_eval."